OPERATING SYSTEMS – UNIT III

Process Synchronization & Deadlocks

SECTION B – 5-MARK QUESTIONS

Each answer prepared in detailed 10-mark format.

Q1. Explain how the Dining Philosophers Problem can be solved using Semaphores.

1. What is Dining Philosophers Problem?

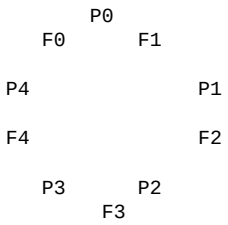
The Dining Philosophers Problem is a process synchronization problem in Operating Systems.

Imagine five philosophers sitting around a circular table.

- Each philosopher has one plate.
- There are five forks.
- Each philosopher needs two forks to eat.
- They think when they are not eating.

The problem is how to share five forks safely without creating deadlock.

2. Diagram



- P0 to P4 = Philosophers.
- F0 to F4 = Forks.

3. Working

1. Initially, all philosophers are thinking.
2. A philosopher becomes hungry.
3. The philosopher picks up the left fork.
4. Then picks up the right fork.
5. After getting both forks, the philosopher eats.
6. After eating, both forks are released.
7. The philosopher starts thinking again.

4. Solution Using Semaphores

A semaphore is used to control access to shared resources.

We use:

- Five binary semaphores for five forks.
- One counting semaphore called `room = 4`.

The room semaphore allows only four philosophers to compete for forks at a time.

5. Algorithm

```
semaphore room = 4;
semaphore fork[5] = {1,1,1,1,1};

Philosopher(i)
{
    wait(room);

    wait(fork[i]);

    wait(fork[(i+1) % 5]);

    EAT;

    signal(fork[i]);

    signal(fork[(i+1) % 5]);

    signal(room);

    THINK;
}
```

6. Step-by-Step Example

1. All five forks are available.
2. Four philosophers can enter the room.
3. Each philosopher tries to collect two forks.
4. A philosopher who gets both forks starts eating.
5. After eating, the philosopher releases both forks.
6. Another philosopher can now use the released forks.

7. Advantages

1. Prevents the deadlock caused by all five philosophers holding one fork.
2. Allows multiple philosophers to eat when possible.
3. Controls resource sharing.
4. Helps understand process synchronization.

8. Disadvantages

1. Some philosophers may wait longer.
2. Starvation is possible without fair scheduling.
3. Semaphore operations must be used correctly.

9. Conclusion

The Dining Philosophers Problem explains how multiple processes share limited resources. Using a counting semaphore with value four helps prevent deadlock.

Remember: 5 Philosophers + 5 Forks + Room = 4.

Q2. Explain Deadlock Avoidance with an example of Banker's Algorithm.

1. What is Deadlock Avoidance?

Deadlock Avoidance is a method used by the Operating System to prevent deadlock before it happens.

The OS checks whether giving resources to a process will keep the system in a Safe State.

Banker's Algorithm is used for deadlock avoidance.

Real-life example: A bank gives loans only when it can safely manage the money needed by all customers.

2. Important Terms

1. **Available:** Free resources available in the system.

2. **Maximum:** Maximum resources required by a process.

3. **Allocation:** Resources already given to a process.

4. **Need:** Additional resources required by a process.

Formula:

\[

Need = Maximum - Allocation

\]

3. Example

Consider five processes P0, P1, P2, P3 and P4.

There are three resource types: A, B and C.

Available Resources:

A	B	C
3	3	2

Allocation Matrix:

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum Matrix:

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

4. Step 1 – Calculate Need Matrix

Formula:

\[

Need = Maximum - Allocation

\]

For P0:

$$
(7,5,3)-(0,1,0)=(7,4,3)
$$

For P1:

$$
(3,2,2)-(2,0,0)=(1,2,2)
$$

For P2:

$$
(9,0,2)-(3,0,2)=(6,0,0)
$$

For P3:

$$
(2,2,2)-(2,1,1)=(0,1,1)
$$

For P4:

$$
(4,3,3)-(0,0,2)=(4,3,1)
$$

Final Need Matrix:

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

5. Step 2 – Find Safe Sequence

Initially:

$$
Work=(3,3,2)
$$

Step 1: Check P1

Need = (1,2,2)

P1 can finish because its need is less than or equal to Work.

New Work:

\[
(3,3,2)+(2,0,0)=(5,3,2)
\]

Step 2: Check P3

Need = (0,1,1)

P3 can finish.

\[
(5,3,2)+(2,1,1)=(7,4,3)
\]

Step 3: Check P4

Need = (4,3,1)

P4 can finish.

\[
(7,4,3)+(0,0,2)=(7,4,5)
\]

Step 4: Check P0

Need = (7,4,3)

P0 can finish.

\[
(7,4,5)+(0,1,0)=(7,5,5)
\]

Step 5: Check P2

Need = (6,0,0)

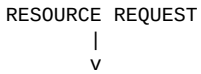
P2 can finish.

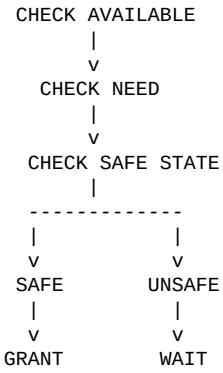
\[
(7,5,5)+(3,0,2)=(10,5,7)
\]

Safe Sequence Found

\[
P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2
\]

6. Diagram





7. Advantages

1. Prevents unsafe resource allocation.
2. Avoids deadlock.
3. Checks resource requirements.
4. Maintains system safety.

8. Disadvantages

1. Maximum resource requirements must be known.
2. Safety checking takes time.
3. Resource allocation may be delayed.

9. Conclusion

Banker's Algorithm checks whether the system is safe before allocating resources. It grants resources only when a safe sequence is possible.

Remember: Need = Maximum – Allocation.

Q3. Explain the Readers-Writers Problem using Semaphores.

1. What is Readers-Writers Problem?

The Readers-Writers Problem is a synchronization problem in which multiple processes access the same shared data.

There are two types of processes:

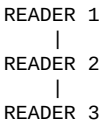
- Reader – Reads data.
- Writer – Changes or writes data.

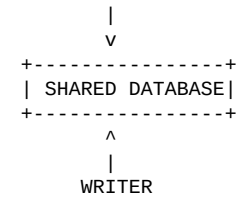
Rules:

1. Multiple readers can read at the same time.
2. Only one writer can write at a time.
3. Readers cannot read while a writer is writing.
4. Writers must wait until readers finish.

Example: Many students can read a notice board, but only one person should edit it at a time.

2. Diagram





3. Semaphores Used

We use two semaphores and one variable.

Variable	Initial value	Purpose
mutex	1	Protects reader count
wrt	1	Controls shared data
read_count	0	Counts readers

4. Reader Algorithm

```
wait(mutex);

read_count++;

if (read_count == 1)
    wait(wrt);

signal(mutex);

READ DATA;

wait(mutex);

read_count--;

if (read_count == 0)
    signal(wrt);

signal(mutex);
```

5. Writer Algorithm

```
wait(wrt);

WRITE DATA;

signal(wrt);
```

6. Step-by-Step Working

1. Reader 1 enters and increases the count to 1.
2. Reader 1 locks the writer semaphore.
3. Reader 2 enters and increases the count to 2.
4. Both readers can read together.
5. Reader 1 finishes and decreases the count to 1.
6. Reader 2 finishes and decreases the count to 0.
7. The writer semaphore is released.
8. Now the writer can write.

7. Advantages

1. Multiple readers can read together.
2. Protects shared data.

3. Prevents reading during writing.

4. Maintains data consistency.

8. Disadvantages

1. Writers may wait for a long time.

2. Starvation may occur.

3. Semaphore management is difficult.

9. Conclusion

The Readers-Writers Problem explains how multiple readers and writers share data safely using semaphores.

Remember: Multiple Readers – One Writer at a Time.

Q4. Discuss the necessary conditions for Deadlock with examples.

1. What is Deadlock?

Deadlock is a situation in which two or more processes wait forever for resources held by each other.

Example: Two students have two books. Student A has Book 1 and wants Book 2. Student B has Book 2 and wants Book 1. Both wait forever.

2. Four Necessary Conditions

Deadlock occurs when all four conditions exist together.

1. Mutual Exclusion

- A resource is used by only one process at a time.
- Example: One printer is used by one process.

2. Hold and Wait

- A process holds one resource and waits for another.
- Example: P1 holds a printer and waits for a scanner.

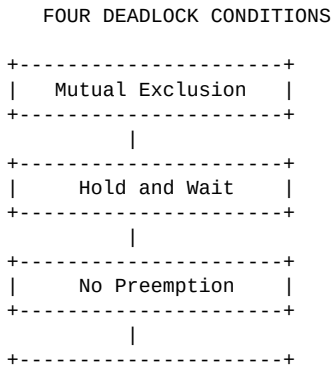
3. No Preemption

- A resource cannot be forcibly taken from a process.
- The process must release it voluntarily.

4. Circular Wait

- Processes wait for resources in a circular order.
- Example: P1 waits for P2 and P2 waits for P1.

3. Diagram



```
|     Circular Wait    |
+----------------------+
```

4. Example of Deadlock

```
P1 ---------> R2
^              |
|              v
R1 <--------- P2
```

- P1 holds R1 and requests R2.
- P2 holds R2 and requests R1.
- Both processes wait forever.

5. Importance

1. Helps identify deadlocks.
2. Helps prevent deadlocks.
3. Improves resource management.
4. Helps maintain system stability.

6. Conclusion

The four necessary conditions are Mutual Exclusion, Hold and Wait, No Preemption and Circular Wait. All four must exist together for deadlock to occur.

Remember: M – H – N – C.

Q5. Define Semaphores and explain how they are used in Process Synchronization.

1. What is a Semaphore?

A Semaphore is a synchronization tool used by the Operating System to control access to shared resources.

It helps processes work safely without interfering with each other.

Example: A parking area has only one free space. A token allows only one car to enter.

2. Types of Semaphores

1. Binary Semaphore

- It has only two values: 0 and 1.
- It allows one process to access a protected resource at a time.
- Example: One printer.

2. Counting Semaphore

- It can have values from zero to a positive number.
- It controls multiple instances of a resource.
- Example: A parking area with five spaces.

3. Semaphore Operations

Wait() Operation

- Used to acquire a resource.
- If the resource is unavailable, the process waits.

```
wait(S)
{
    while (S <= 0)
        wait;

    S--;
}
```

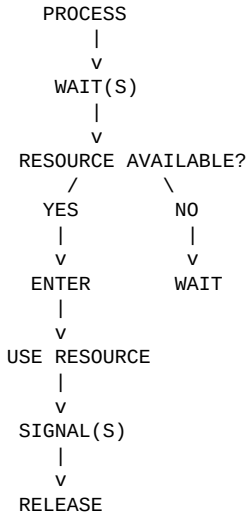
Signal() Operation

- Used to release a resource.
- It increases the semaphore value and may wake a waiting process.

```
signal(S)
{
    S++;
}
```

These operations must be atomic.

4. Diagram



5. Example – Printer

Initially:

\[

S=1

\]

1. P1 requests the printer.
2. P1 executes wait(S).
3. S becomes 0.
4. P1 uses the printer.
5. P2 requests the printer and waits.
6. P1 finishes printing.
7. P1 executes signal(S).
8. P2 can now use the printer.

6. Advantages

1. Prevents race conditions.
2. Controls shared resources.

3. Supports process synchronization.

4. Coordinates multiple processes.

7. Disadvantages

1. Incorrect use can cause deadlocks.

2. Starvation may occur.

3. Difficult to manage in large programs.

8. Conclusion

A semaphore is an important synchronization tool. It uses Wait and Signal operations to control access to shared resources.

Remember: Wait = Acquire, Signal = Release.

Q6. Write short notes on Peterson's Solution for the Critical Section Problem.

1. What is Peterson's Solution?

Peterson's Solution is a software method used to solve the Critical Section Problem for two processes.

It ensures that only one process enters the critical section at a time.

It uses two shared variables:

1. Flag.

2. Turn.

2. Important Variables

Flag Variable

- Shows whether a process wants to enter.
- `flag[i] = true` means process Pi wants to enter.

Turn Variable

- Decides whose turn it is.
- Helps two processes coordinate.

3. Algorithm

```
Process Pi:

flag[i] = true;

turn = j;

while (flag[j] && turn == j)
    wait;

/* Critical Section */

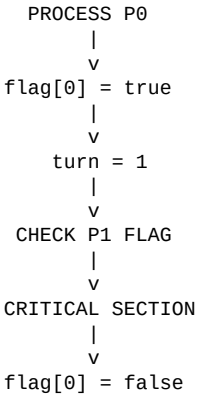
flag[i] = false;

/* Remainder Section */
```

Here:

- i = Current process.
- j = Other process.

4. Diagram



5. Step-by-Step Working

1. P0 wants to enter the critical section.
2. P0 sets `flag[0] = true`.
3. P0 gives the turn to P1.
4. P0 checks whether P1 also wants to enter.
5. If P1 does not want to enter, P0 enters.
6. P0 completes its work.
7. P0 sets `flag[0] = false`.
8. P1 can enter when its conditions allow.

6. Three Requirements

1. Mutual Exclusion

Only one process can enter at a time.

2. Progress

A process should not wait unnecessarily when the critical section is free.

3. Bounded Waiting

A process should not wait forever.

7. Advantages

1. Simple software solution.
2. Explains process synchronization.
3. Provides mutual exclusion for two processes.

8. Disadvantages

1. Works for only two processes in its basic form.
2. Uses busy waiting.
3. Modern systems require proper memory-ordering support.

9. Conclusion

Peterson's Solution uses Flag and Turn variables to control two processes and solve the Critical Section Problem.

Remember: Peterson = Two Processes + Two Variables.

Q7. Explain the Critical Section Problem with an example.

1. What is a Critical Section?

A Critical Section is a part of a program where a process accesses shared data or shared resources.

When multiple processes access the same data at the same time, incorrect results may occur.

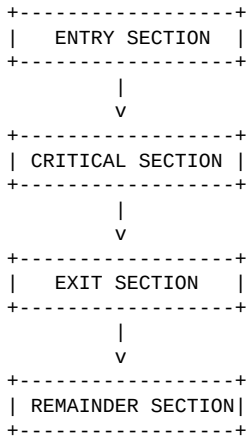
This is called a Race Condition.

Example: Two people withdrawing money from the same bank account simultaneously.

2. Four Parts of a Process

1. Entry Section.
2. Critical Section.
3. Exit Section.
4. Remainder Section.

3. Diagram



4. Explanation of Four Sections

1. Entry Section

- A process requests permission to enter.
- It checks whether another process is using the resource.

2. Critical Section

- The process accesses shared data.
- It performs reading or writing operations.

3. Exit Section

- The process releases the shared resource.
- Another process can enter.

4. Remainder Section

- The process performs other operations.
- It does not access the protected resource.

5. Example – Bank Account

Suppose the bank account balance is ■1,000.

Two processes try to withdraw ■500 simultaneously.

Without synchronization:

1. P1 reads ■1,000.
2. P2 also reads ■1,000.
3. P1 subtracts ■500.
4. P2 subtracts ■500.
5. Both may write ■500.

The final balance becomes incorrectly ■500 instead of ■0.

6. Three Requirements

1. Mutual Exclusion: Only one process enters the critical section at a time.

2. Progress: If the critical section is free, a process should be allowed to enter without unnecessary delay.

3. Bounded Waiting: A process should not wait forever.

7. Methods to Solve the Problem

1. Peterson's Solution.
2. Mutex Locks.
3. Semaphores.
4. Monitors.

8. Advantages

1. Prevents race conditions.
2. Protects shared data.
3. Maintains data consistency.
4. Coordinates multiple processes.

9. Conclusion

The Critical Section Problem occurs when multiple processes access shared resources. Mutual Exclusion, Progress and Bounded Waiting are the three important requirements.

Remember: Entry → Critical → Exit → Remainder.

Q8. What are Mutex Locks? Explain their usage with an example.

1. What is a Mutex Lock?

A Mutex Lock is a synchronization mechanism used to protect shared resources.

It allows only one process or thread to access the protected critical section at a time.

Mutex means Mutual Exclusion.

Example: A bathroom has one key. Only the person holding the key can use it.

2. Working of Mutex Lock

A mutex has two main operations:

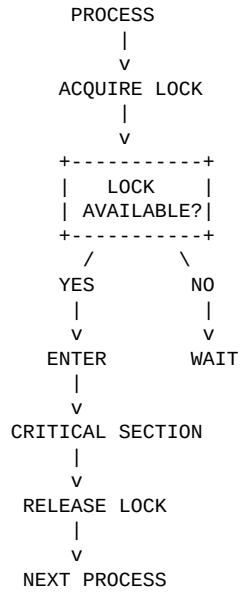
1. Acquire()

- Locks the resource.
- Other processes must wait.

2. Release()

- Unlocks the resource.
- Another process can use it.

3. Diagram



4. Example – Shared Printer

1. Initially, the mutex is unlocked.
2. P1 requests the printer.
3. P1 acquires the mutex lock.
4. P1 starts printing.
5. P2 requests the printer but waits.
6. P1 finishes printing.
7. P1 releases the lock.
8. P2 acquires the lock and starts printing.

5. Pseudocode

```
while (true)
{
    acquire(lock);

    // Critical Section
    USE SHARED RESOURCE;

    release(lock);

    // Remainder Section
}
```

6. Advantages

1. Prevents simultaneous access.
2. Avoids race conditions.

3. Protects shared resources.

4. Easy to understand.

7. Disadvantages

1. Processes may wait.

2. Incorrect locking can cause deadlocks.

3. Some implementations may use busy waiting.

8. Conclusion

Mutex Locks protect shared resources by allowing only one process to hold the lock at a time.

Remember: Acquire = Lock, Release = Unlock.

Q9. What is a Semaphore? Explain the Producer-Consumer Problem using Semaphores.

1. What is Producer-Consumer Problem?

The Producer-Consumer Problem is a process synchronization problem.

It contains three main parts:

- Producer – Creates items.
- Consumer – Uses items.
- Buffer – Stores items temporarily.

Example: A bakery produces bread and stores it in a box. Customers take bread from the box.

2. Working

1. The producer creates an item.

2. The producer puts it into the buffer.

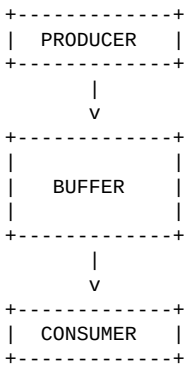
3. The consumer takes the item.

4. The consumer uses it.

5. The producer waits if the buffer is full.

6. The consumer waits if the buffer is empty.

3. Diagram



4. Semaphores Used

We use three semaphores.

Semaphore	Initial Value	Purpose
mutex	1	Protects buffer
empty	N	Counts empty slots
full	0	Counts filled slots

Here N is the buffer size.

5. Producer Algorithm

```
while (true)
{
    PRODUCE ITEM;

    wait(empty);
    wait(mutex);

    ADD ITEM TO BUFFER;

    signal(mutex);
    signal(full);
}
```

6. Consumer Algorithm

```
while (true)
{
    wait(full);
    wait(mutex);

    REMOVE ITEM FROM BUFFER;

    signal(mutex);
    signal(empty);

    CONSUME ITEM;
}
```

7. Step-by-Step Example

Assume buffer size = 3.

Initially:

- Empty = 3.
- Full = 0.

Step 1: Producer adds item A.

- Empty = 2.
- Full = 1.

Step 2: Producer adds item B.

- Empty = 1.
- Full = 2.

Step 3: Producer adds item C.

- Empty = 0.
- Full = 3.

Now the buffer is full. Producer must wait.

Step 4: Consumer removes item A.

- Empty = 1.

- Full = 2.

Now the producer can add another item.

8. Advantages

1. Prevents buffer overflow.
2. Prevents buffer underflow.
3. Maintains synchronization.
4. Protects shared buffer access.

9. Conclusion

The Producer-Consumer Problem explains how two processes share a common buffer. Semaphores control access and prevent incorrect operations.

Remember: Producer = Adds, Consumer = Removes.

Q10. What is a Monitor? Explain with an example how monitors achieve synchronization.

1. What is a Monitor?

A Monitor is a high-level synchronization mechanism used to control access to shared resources.

It combines shared data and functions into one protected structure.

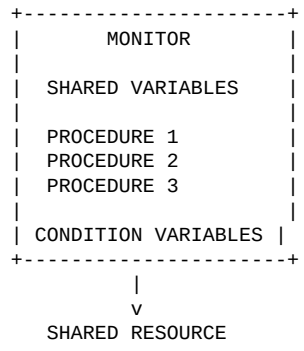
Only one process or thread can execute inside a monitor at a time.

Example: A bank counter allows only one customer to perform a transaction at a particular counter at a time.

2. Main Components

1. Shared Variables – Store common data.
2. Procedures – Functions that access shared data.
3. Initialization Code – Sets starting values.
4. Condition Variables – Allow processes to wait and continue.

3. Diagram



4. Condition Variables

wait()

- Makes a process wait.
- Releases the monitor so another process can enter.

signal()

- Wakes a waiting process when the required condition is signalled.

5. Example – Producer-Consumer

```
monitor Buffer
{
    condition notFull;
    condition notEmpty;

    insert(item)
    {
        while (buffer is full)
            wait(notFull);

        ADD ITEM;

        signal(notEmpty);
    }

    remove()
    {
        while (buffer is empty)
            wait(notEmpty);

        REMOVE ITEM;

        signal(notFull);
    }
}
```

6. Step-by-Step Working

1. Producer enters the monitor.
2. Producer checks whether the buffer has space.
3. If the buffer is full, producer waits.
4. Consumer enters and removes an item.
5. Consumer signals `notFull`.
6. Producer wakes up and adds an item.

7. Advantages

1. Provides automatic mutual exclusion.
2. Makes synchronization easier.
3. Organizes shared data and functions.
4. Reduces locking mistakes.

8. Disadvantages

1. Requires programming language or runtime support.
2. Incorrect condition handling can cause problems.
3. Fair scheduling may need additional implementation.

9. Conclusion

A Monitor is a high-level synchronization tool that combines shared data, procedures and condition variables.

Remember: Monitor = Shared Data + Procedures + Condition Variables.

Q11. Explain Deadlock Prevention and discuss the methods used to prevent deadlocks.

1. What is Deadlock Prevention?

Deadlock Prevention is a method used by the Operating System to stop deadlock before it happens.

It works by breaking at least one of the four necessary conditions of deadlock.

2. Four Conditions

1. Mutual Exclusion.
2. Hold and Wait.
3. No Preemption.
4. Circular Wait.

3. Methods of Deadlock Prevention

1. Prevent Mutual Exclusion

- Make resources shareable wherever possible.
- Multiple processes can use a shared resource.
- Example: Multiple processes can read the same file.

2. Prevent Hold and Wait

- A process must request all required resources before execution.
- Or it must release existing resources before requesting new resources.
- Example: Request printer and scanner together.

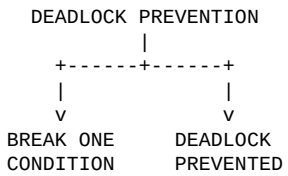
3. Prevent No Preemption

- The OS can take resources away when it is safe.
- Example: CPU time can be taken away from a process.

4. Prevent Circular Wait

- Give every resource a unique number.
- Processes must request resources in increasing order.
- This prevents circular waiting.

4. Diagram



5. Advantages

1. Prevents deadlock before it occurs.
2. Makes resource allocation safer.
3. Improves system management.
4. Reduces deadlock problems.

6. Disadvantages

1. Resource utilization may decrease.
2. Processes may wait longer.

3. Some resources cannot be taken back.

4. Requesting all resources in advance may be difficult.

7. Conclusion

Deadlock Prevention stops deadlock by breaking at least one of its four necessary conditions.

Remember: Break Any One Condition = No Deadlock.

Q12. Explain Deadlock Detection for systems with single and multiple instances of resource types.

1. What is Deadlock Detection?

Deadlock Detection is a method used by the Operating System to find whether deadlock has occurred.

Unlike prevention, this method allows deadlock to happen and then checks for it.

There are two cases:

1. Single Instance of Each Resource Type.

2. Multiple Instances of Resource Types.

2. Single Instance of Each Resource Type

When each resource type has only one instance, the OS uses a Resource Allocation Graph.

It contains:

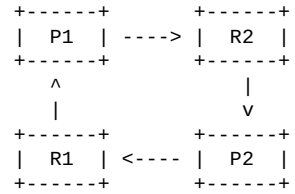
- Process nodes.
- Resource nodes.
- Request edges.
- Allocation edges.

Rules:

- Process → Resource means the process requests a resource.
- Resource → Process means the resource is allocated.

If the graph contains a cycle, deadlock exists when every resource type has only one instance.

3. Diagram



Here:

- P1 waits for R2.
- P2 waits for R1.
- Both are waiting forever.

4. Multiple Instances of Resource Types

When a resource type has multiple instances, the OS uses a Deadlock Detection Algorithm.

It uses three data structures:

1. Available – Free resources.

2. Allocation – Resources already given.

3. Request – Additional resources required.

5. Detection Algorithm

1. Set Work = Available.

2. Find a process whose Request is less than or equal to Work.

3. Assume that process finishes.

4. Add its allocated resources to Work.

5. Repeat the process.

6. If some processes cannot finish, they may be deadlocked.

6. Example

Available:

\[

Work=(0,0)

\]

Allocation:

Process	A	B
P1	1	0
P2	0	1

Request:

Process	A	B
P1	0	1
P2	1	0

Step 1: Check P1.

P1 needs (0,1), but Work = (0,0).

P1 cannot continue.

Step 2: Check P2.

P2 needs (1,0), but Work = (0,0).

P2 cannot continue.

Step 3: Both processes are unable to finish.

Deadlock Detected

7. Difference Between Single and Multiple Instances

Single Instance	Multiple Instances
One resource instance.	More than one instance.

Single Instance	Multiple Instances
Uses Resource Allocation Graph.	Uses detection algorithm.
Cycle indicates deadlock.	Cycle alone may not prove deadlock.
Easy to check.	Requires calculations.

8. Conclusion

Deadlock Detection helps the OS find deadlocked processes. A Resource Allocation Graph is used for single-instance resources, and an algorithm using matrices is used for multiple instances.

Remember: Single = Graph; Multiple = Algorithm.

Q13. Discuss the methods of Recovery from Deadlock, including Process Termination and Resource Preemption.

1. What is Deadlock Recovery?

Deadlock Recovery is the process of removing a deadlock after it has been detected.

When processes are deadlocked, they cannot continue execution.

The OS uses two main methods:

1. Process Termination.

2. Resource Preemption.

2. Process Termination

In this method, the OS stops one or more processes to remove deadlock.

There are two ways.

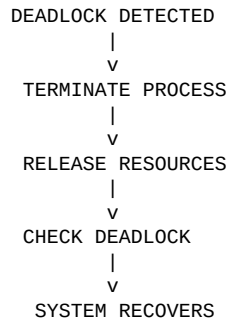
A. Terminate All Processes

- The OS stops all deadlocked processes.
- Their resources are released.
- The system can continue.

B. Terminate One Process at a Time

- The OS stops one process.
- Checks whether deadlock is removed.
- If deadlock still exists, another process is stopped.
- This continues until deadlock disappears.

3. Diagram



4. Resource Preemption

Resource Preemption means taking a resource away from one process and giving it to another process.

Example: P1 holds a resource. The OS takes that resource and gives it to P2 so P2 can finish.

5. Steps in Resource Preemption

1. Select a process.
2. Take its resource.
3. Give the resource to another process.
4. Allow the other process to continue.
5. Restart the affected process later if required.

6. Important Issues

1. Victim Selection

The OS decides which process should lose its resource.

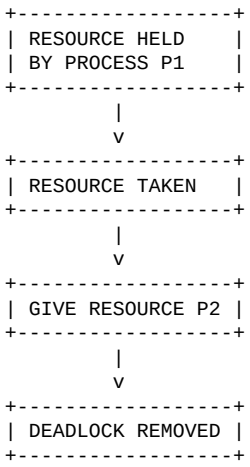
2. Rollback

The affected process may be returned to an earlier safe state.

3. Starvation

The same process should not repeatedly lose resources.

7. Diagram



8. Difference Between Recovery Methods

Process Termination	Resource Preemption
Stops processes.	Takes resources away.
Releases process resources.	Reallocates resources.
Work may be lost.	Rollback may be needed.
Simple method.	Requires careful selection.

9. Conclusion

Deadlock Recovery removes existing deadlocks. The OS uses Process Termination or Resource Preemption to allow the system to continue.

Remember: Termination = Stop Process; Preemption = Take Resource.

Q14. Explain the different methods used for handling Deadlocks, highlighting the role of each method.

1. What is Deadlock Handling?

Deadlock Handling means using different methods to manage and control deadlock in an Operating System.

There are four main methods:

1. Deadlock Prevention.

2. Deadlock Avoidance.

3. Deadlock Detection and Recovery.

4. Ignore Deadlock.

2. Deadlock Prevention

- Stops deadlock before it happens.
- Breaks at least one necessary condition.
- Example: Resource ordering.

3. Deadlock Avoidance

- Checks whether resource allocation is safe.
- Grants resources only if the system remains safe.
- Example: Banker's Algorithm.

4. Deadlock Detection and Recovery

- Allows deadlock to happen.
- Checks whether deadlock exists.
- Removes deadlock using termination or preemption.

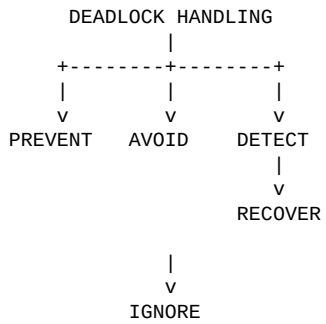
Example: Resource Allocation Graph for single-instance resource types.

5. Ignore Deadlock

- The OS does not actively prevent or detect deadlock.
- It assumes deadlocks are rare.
- It handles them only when necessary.

This is also called the Ostrich Approach.

6. Diagram



7. Comparison

Method	Purpose	Example
Prevention	Stops deadlock conditions	Resource ordering
Avoidance	Maintains safe state	Banker's Algorithm
Detection	Finds deadlock	Resource Allocation Graph
Recovery	Removes deadlock	Termination
Ignore	No active handling	Ostrich Approach

8. Advantages

1. Helps manage system resources.
2. Reduces deadlock problems.
3. Improves system stability.
4. Provides different methods for different systems.

9. Conclusion

The Operating System handles deadlocks using Prevention, Avoidance, Detection and Recovery. Some systems also ignore rare deadlocks when active handling is not worthwhile.

Remember:

- Prevention = Stop.
- Avoidance = Stay Safe.
- Detection = Find.
- Recovery = Remove.
- Ignore = Do Nothing Actively.

SECTION C – 10-MARK COMPREHENSIVE ESSAY QUESTIONS

Q15. Explain the classic problems of Process Synchronization (Bounded Buffer, Readers-Writers and Dining Philosophers) in detail using Semaphores and Monitors. Provide neat sketches and comparative analysis.

1. What is Process Synchronization?

Process Synchronization is a method used by the Operating System to coordinate multiple processes.

It makes sure that processes share resources safely without creating problems.

Example: Two people should not change the same bank account balance at exactly the same time without proper control.

There are three classic synchronization problems:

1. Bounded Buffer Problem.
2. Readers-Writers Problem.
3. Dining Philosophers Problem.

2. Bounded Buffer Problem (Producer-Consumer)

What is it?

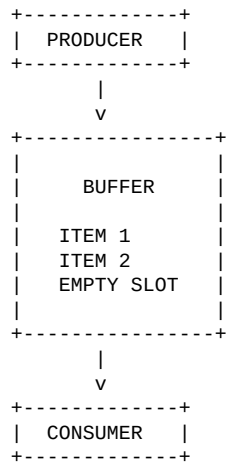
The Bounded Buffer Problem explains how a Producer and Consumer share a limited storage area.

- Producer creates items.
- Buffer stores items.
- Consumer takes items.

Working:

1. Producer creates an item.
2. Producer adds it to the buffer.
3. Consumer removes the item.
4. Consumer uses the item.
5. Producer waits if the buffer is full.
6. Consumer waits if the buffer is empty.

Diagram:



3. Bounded Buffer Using Semaphores

Three semaphores are used.

Semaphore	Initial Value	Purpose
mutex	1	Protects buffer
empty	N	Counts empty slots
full	0	Counts filled slots

Producer Algorithm:

```
while (true)
{
    PRODUCE ITEM;

    wait(empty);
    wait(mutex);

    INSERT ITEM;

    signal(mutex);
    signal(full);
}
```

Consumer Algorithm:

```
while (true)
{
    wait(full);
    wait(mutex);

    REMOVE ITEM;
```

```
    signal(mutex);
    signal(empty);

    CONSUME ITEM;
}
```

4. Bounded Buffer Using Monitor

A Monitor combines shared data and operations into one protected structure.

```
monitor Buffer
{
    condition notFull;
    condition notEmpty;

    insert(item)
    {
        while (buffer is full)
            wait(notFull);

        ADD ITEM;

        signal(notEmpty);
    }

    remove()
    {
        while (buffer is empty)
            wait(notEmpty);

        REMOVE ITEM;

        signal(notFull);
    }
}
```

5. Readers-Writers Problem

What is it?

The Readers-Writers Problem explains how multiple processes can access shared data safely.

There are two types:

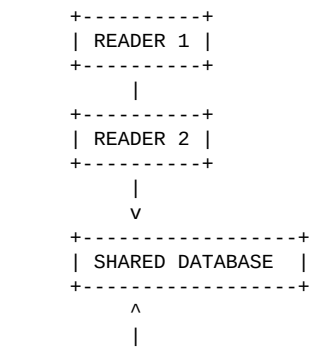
- Reader – Reads data.
- Writer – Writes or changes data.

Working:

1. Multiple readers can read together.
2. Only one writer can write at a time.
3. A reader cannot read while a writer is writing.
4. A writer must wait until readers finish.

Example: Many students can read a notice board, but only one person should edit it.

Diagram:



```
+----------+
|  WRITER  |
+----------+
```

6. Readers-Writers Using Semaphores

We use:

- mutex = 1.
- wrt = 1.
- read_count = 0.

Reader Algorithm:

```
wait(mutex);

read_count++;

if (read_count == 1)
    wait(wrt);

signal(mutex);

READ DATA;

wait(mutex);

read_count--;

if (read_count == 0)
    signal(wrt);

signal(mutex);
```

Writer Algorithm:

```
wait(wrt);

WRITE DATA;

signal(wrt);
```

This reader-preference solution may cause writer starvation.

7. Readers-Writers Using Monitor

A Monitor uses condition variables to control readers and writers.

```
monitor RW
{
    int readers = 0;
    bool writing = false;

    startRead()
    {
        while (writing)
            wait(canRead);

        readers++;
    }

    endRead()
    {
        readers--;

        if (readers == 0)
            signal(canWrite);
    }

    startWrite()
    {
        while (writing || readers > 0)
            wait(canWrite);

        writing = true;
```

```
    }

    endWrite()
    {
        writing = false;

        signal(canWrite);
        signal(canRead);
    }
}
```

This is simplified monitor pseudocode. A fair waiting policy is needed to avoid starvation.

8. Dining Philosophers Problem

What is it?

The Dining Philosophers Problem explains how multiple processes share limited resources.

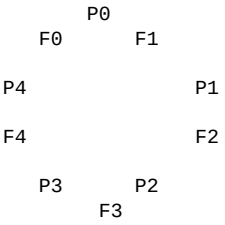
Imagine five philosophers sitting around a circular table.

- Each philosopher has one plate.
- Five forks are available.
- Each philosopher needs two forks to eat.

Working:

1. Philosophers think.
2. A philosopher becomes hungry.
3. The philosopher picks up two forks.
4. The philosopher eats.
5. After eating, both forks are released.

Diagram:



9. Dining Philosophers Using Semaphores

We use:

- Five binary semaphores for five forks.
- One counting semaphore `room = 4`.

```
semaphore room = 4;
semaphore fork[5] = {1,1,1,1,1};

Philosopher(i)
{
    wait(room);

    wait(fork[i]);

    wait(fork[(i+1) % 5]);

    EAT;

    signal(fork[i]);

    signal(fork[(i+1) % 5]);
```

```
    signal(room);

    THINK;
}
```

The room semaphore allows only four philosophers to compete for forks at a time.

10. Dining Philosophers Using Monitor

A Monitor checks the state of each philosopher.

Three states are used:

1. THINKING.

2. HUNGRY.

3. EATING.

A philosopher can eat only when both neighboring philosophers are not eating.

```
monitor Dining
{
    state[5] = THINKING;

    test(i)
    {
        if (state[i] == HUNGRY
            && left neighbor != EATING
            && right neighbor != EATING)
        {
            state[i] = EATING;
            signal(self[i]);
        }
    }
}
```

11. Comparison of Three Problems

Problem	Shared Resource	Main Problem
Bounded Buffer	Buffer	Full or empty buffer
Readers-Writers	Database	Reading and writing conflict
Dining Philosophers	Forks	Deadlock and starvation

12. Semaphore vs Monitor

Semaphore	Monitor
Low-level tool	High-level structure
Uses Wait and Signal	Uses procedures and conditions
Programmer controls locking	Automatic mutual exclusion
More difficult to manage	Easier to organize
Flexible	Requires language/runtime support

13. Advantages

1. Prevents race conditions.

2. Protects shared resources.

3. Coordinates multiple processes.

4. Maintains data consistency.

14. Conclusion

The three classic synchronization problems explain how processes share resources safely. Semaphores and Monitors help control access and prevent synchronization problems.

Remember:

- Bounded Buffer = Producer and Consumer.
- Readers-Writers = Reading and Writing.
- Dining Philosophers = Sharing Forks.

Q16. Explain Deadlock Characterization, Prevention, Avoidance (Banker's Algorithm), and Detection & Recovery in detail with neat diagrams and comparative analysis.

1. What is Deadlock?

Deadlock is a situation in which two or more processes wait forever for resources held by each other.

Example: Two students hold different books. Both need the other's book to continue their work. Neither releases the book, so both wait forever.

2. Deadlock Characterization

Deadlock occurs when four necessary conditions exist together.

1. Mutual Exclusion

- One resource can be used by only one process at a time.
- Example: One printer.

2. Hold and Wait

- A process holds one resource and waits for another.
- Example: P1 holds a printer and waits for a scanner.

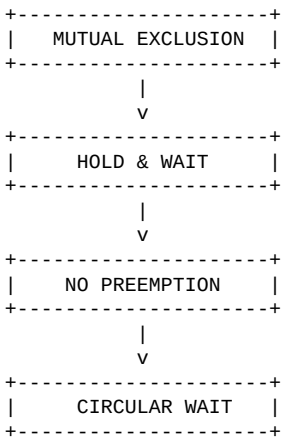
3. No Preemption

- A resource cannot be forcibly taken from a process.
- The process must release it voluntarily.

4. Circular Wait

- Processes wait for resources in a circular chain.
- Example: P1 waits for P2 and P2 waits for P1.

Diagram:



3. Deadlock Prevention

What is it?

Deadlock Prevention stops deadlock before it happens by breaking at least one of the four necessary conditions.

Methods:

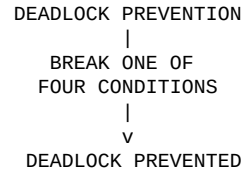
1. Prevent Mutual Exclusion – Make resources shareable wherever possible.

2. Prevent Hold and Wait – Request all required resources together or release existing resources before requesting more.

3. Prevent No Preemption – Take resources away when it is safe.

4. Prevent Circular Wait – Give resources a fixed order and request them in that order.

Diagram:



4. Deadlock Avoidance

What is it?

Deadlock Avoidance checks whether giving resources to a process will keep the system in a Safe State.

The most important algorithm is Banker's Algorithm.

5. Banker's Algorithm – Important Terms

Term	Meaning
Available	Free resources
Maximum	Maximum resources required
Allocation	Resources already given
Need	Additional resources required

Formula:

\[
Need = Maximum - Allocation
\]

6. Numerical Problem – Banker's Algorithm

Given:

Five processes: P0, P1, P2, P3, P4.

Three resource types: A, B, C.

Available:

\[
(3,3,2)
\]

Allocation Matrix:

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum Matrix:

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Step 1 – Calculate Need Matrix

Formula:

\[
Need = Maximum - Allocation
\]

P0:

\[
(7,5,3)-(0,1,0)=(7,4,3)
\]

P1:

\[
(3,2,2)-(2,0,0)=(1,2,2)
\]

P2:

\[
(9,0,2)-(3,0,2)=(6,0,0)
\]

P3:

\[
(2,2,2)-(2,1,1)=(0,1,1)
\]

P4:

\[

(4,3,3)-(0,0,2)=(4,3,1)

\]

Final Need Matrix:

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Step 2 – Find Safe Sequence

Initially:

\[

Work=(3,3,2)

\]

Check P1:

Need = (1,2,2)

P1 can finish.

\[

Work=(3,3,2)+(2,0,0)

\]

\[

Work=(5,3,2)

\]

Check P3:

Need = (0,1,1)

P3 can finish.

\[

Work=(5,3,2)+(2,1,1)

\]

\[

Work=(7,4,3)

\]

Check P4:

Need = (4,3,1)

P4 can finish.

\[

Work=(7,4,3)+(0,0,2)

\]

\[

Work=(7,4,5)

\]

Check P0:

Need = (7,4,3)

P0 can finish.

\[

Work=(7,4,5)+(0,1,0)

\]

\[

Work=(7,5,5)

\]

Check P2:

Need = (6,0,0)

P2 can finish.

\[

Work=(7,5,5)+(3,0,2)

\]

\[

Work=(10,5,7)

\]

Final Safe Sequence

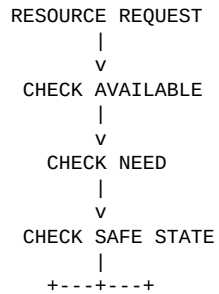
\[

P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2

\]

Therefore, the system is in a Safe State.

7. Banker's Algorithm Diagram



```
  |       |
  v       v
SAFE    UNSAFE
  |       |
  v       v
GRANT    WAIT
```

8. Deadlock Detection

What is it?

Deadlock Detection is a method used to find whether deadlock has already occurred.

There are two cases.

A. Single Instance of Each Resource

- Uses Resource Allocation Graph.
- A cycle indicates deadlock.

Example:

```
P1 ---> R2
^       |
|       v
R1 <--- P2
```

P1 waits for R2, and P2 waits for R1.

B. Multiple Instances of Resources

Uses three data structures:

1. Available.
2. Allocation.
3. Request.

Steps:

1. Set Work = Available.
2. Find a process whose Request is less than or equal to Work.
3. Assume that process finishes.
4. Add its allocated resources to Work.
5. Repeat until no more processes can finish.
6. Unfinished processes are deadlocked under the algorithm's assumptions.

9. Deadlock Recovery

What is it?

Deadlock Recovery removes deadlock after it has been detected.

There are two methods.

1. Process Termination

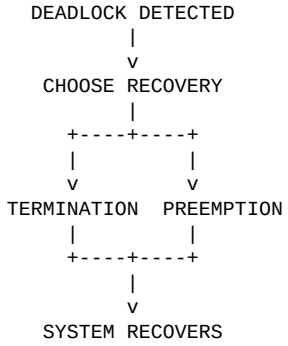
- Stop all deadlocked processes, or
- Stop one process at a time until deadlock disappears.

2. Resource Preemption

- Take a resource from one process.
- Give it to another process.

• Restart the affected process later if necessary.

Diagram:



10. Comparative Analysis

Method	Purpose	Example
Prevention	Stops deadlock conditions	Resource ordering
Avoidance	Keeps system safe	Banker's Algorithm
Detection	Finds deadlock	Resource Allocation Graph
Recovery	Removes deadlock	Termination / Preemption

11. Advantages

1. Helps manage system resources.
2. Reduces deadlock problems.
3. Improves system stability.
4. Helps processes complete execution.
5. Provides different ways to manage resources.

12. Conclusion

Deadlock is an important problem in Operating Systems. It can be handled using Prevention, Avoidance, Detection and Recovery.

Banker's Algorithm is used for avoidance, Resource Allocation Graphs help in detection, and Termination or Preemption helps in recovery.

Remember:

• Prevention = Stop.

• Avoidance = Stay Safe.

• Detection = Find.

• Recovery = Remove.